LAPORAN TUGAS

PEMROGRAMAN BERBASIS JAVA

SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS

Disusun Oleh:

Elena Dementieva	2509116008
Vidya Khansa Mizan	2509116052
Yahya Jailani	2509116085
Rifaa Zainul Arifin	2509116092

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

2026

DAFTAR ISI

DAFTAR ISI ii
DAFTAR GAMBAR iv
DAFTAR TABEL vi
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan 2
BAB II LANDASAN TEORI 3
2.1 Struktur Data 3
2.2 Array 3
2.2.1 Array 1 di mensi 3
2.2.2 Array 2 dimensi 3
2.2.3 Array 3 dimensi 3
2.3 ArrayList 3
2.4 Stack 4
2.5 Queue 4
2.6 Ringkasan Perbandingan 5
BAB III PERANCANGAN PROGRAM 6
3.1 Gambaran Sistem 6
3.2 Pemetaan kebutuhan ke struktur data 6
3.3 Struktur Program 6
3.4 Rancangan Menu 7
3.5 Alur Proses 8
3.5.1 Alur Peminjaman Buku 8
3.5.2 Alur Pembatalan Transaksi (*Undo*) 8
3.5.3 Alur Antrean 9

3.5.4 Aturan Validasi Input 9
BAB IV IMPLEMENTASI DAN PEMBAHASAN 10
4.1 Lingkungan Pengembangan 10
4.2 Implementasi Array 1 Dimensi : Kategori Buku 10
4.3 Implementasi Array 2 Dimensi: Statistik Peminjaman 11
4.4 Implementasi Array 3 Dimensi: Lokasi Penyimpanan 14
4.5 Implementasi ArrayList: Pengelolaan Buku 16
4.5.1 Menambah Buku 16
4.5.2 Menampilkan dan Mencari Buku 18
4.6 Implementasi Stack: Riwayat Transaksi 19
4.7 Implementasi Queue: Antrean Mahasiswa 21
4.8 Implementasi Validasi Input 23
BAB V PENGUJIAN 26
BAB VI PENUTUP 28
6.1 Kesimpulan 28
6.2 Saran 28

DAFTAR GAMBAR

Gambar 3. 1 Tampilan menu utama 8
Gambar 3. 2 Aturan validasi input 9
Gambar 3. 3 Lingkungan pengembangan 10
Gambar 4. 1 perintah CLI 10
Gambar 4. 2 array ID : kategori buku 11
Gambar 4. 3 tampilKategori : buku 11
Gambar 4. 4 Tampilan menu kategori buku 11
Gambar 4. 5 array 2D: jumlah peminjaman 12
Gambar 4. 6 static boolean pinjam 12
Gambar 4. 7 total kategori 13
Gambar 4. 8 output dari statistik peminjaman mingguan 13
Gambar 4. 9 static final lokasi penyimpanan 14
Gambar 4. 10 static boolean taruhDiRak 14
Gambar 4. 11 static void cariLokasi 15
Gambar 4. 12 peta rak per lantai 15
Gambar 4. 13 Hasil pencarian lokasi buku B003 16
Gambar 4. 14 arraylist: daftar buku 16
Gambar 4. 15 static void tambahBuku 17
Gambar 4. 16 output menambah buku 18
Gambar 4. 17 Tampilan daftar buku 18
Gambar 4. 18 hasil pencarian dengan kata kunci “dasar” 19
Gambar 4. 19 static: riwayat transaksi 19
Gambar 4. 20 peminjaman buku B007 19
Gambar 4. 21 pengembalian buku B007 20
Gambar 4. 22 static void lihatTerakhr 20
Gambar 4. 23 riwayat transaksi 20
Gambar 4. 24 static void batalkanTerakhir 21
Gambar 4. 25 pembatalan transaksi terakhir(pengembalian) 21
Gambar 4. 26 queue: antrean mahasiswa 21

Gambar 4. 27 static void tambahAntrean 22
Gambar 4. 28 static void layaniAntrean 22
Gambar 4. 29 output antrean..... 23
Gambar 4. 30 static string inputJudul 24
Gambar 4. 31 static string inputNim..... 24
Gambar 4. 32 validasi input NIM dan ID buku 25
Gambar 4. 33 Hasil pengujian 26

DAFTAR TABEL

Tabel 2. 1 Perbandingan struktur data yang digunakan 5
Tabel 3. 1 Pemetaan kebutuhan program ke struktur data 6
Tabel 3. 2 Struktur package dan kelas 7
Tabel 3. 3 Rancangan menu program 7

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perpustakaan kampus menyimpan banyak buku dari berbagai bidang ilmu, jika pencatatan koleksi buku, lokasi penyimpanan dan juga peminjaman buku masih dilakukan secara manual, petugas akan kesulitan mengetahui di mana letak sebuah buku, siapa yang sedang meminjamnya, dan buku dari kategori apa saja yang paling sering dipinjam. Mahasiswa juga sering kali tidak teratur ketika banyak yang ingin meminjam buku yang sama yang disebabkan oleh tidak adanya catatan siapa yang datang lebih dulu.

Untuk membantu hal tersebut dibutuhkan program sederhana yang dapat mengelola data buku, lokasi penyimpanan, peminjaman, dan antrean. Sehingga dengan melihat banyak masalah yanng terjadi ini kami membuat program dengan menggunakan bahasa pemrograman *java* dan menerapkan enam struktur data. Setiap struktur data kami pakai untuk bagian pekerjaan yang paling sesuai dengan sifatnya. Misalnya *array* dipakai untuk data yang jumlahnya tetap, *ArrayList* untuk daftar buku yang bisa bertambah dan berkurang, *Stack* untuk riwayat transaksi yang bisa dibatalkan dari yang paling akhir, dan *Queue* untuk antrean yang dilayani sesuai urutan datang.

1.2 Rumusan Masalah

Dari latar belakang di atas, maka dapat dirumuskan beberapa masalah yaitu;

1. Bagaimana menyimpan daftar kategori buku menggunakan *array* 1 dimensi?
2. Bagaimana menyimpan jumlah peminjaman buku berdasarkan kategori dan hari dalam satu minggu menggunakan *array* 2 dimensi?
3. Bagaimana merepresentasikan lokasi penyimpanan buku berdasarkan lantai, nomor rak, dan posisi buku menggunakan *array* 3 dimensi?
4. Bagaimana mengelola daftar buku secara dinamis (tambah, tampil, cari, dan hapus) menggunakan *arrayList*?
5. Bagaimana menyimpan riwayat transaksi sehingga transaksi terakhir dapat dilihat dan dibatalkan menggunakan *stack*?

6. Bagaimana mengelola antrean mahasiswa yang ingin meminjam buku berdasarkan urutan kedatangan menggunakan *queue*?

1.3 Tujuan

Dari adanya rumusan masalah diatas, maka dapat disimpulkan beberapa tujuan yaitu;

1. Membuat program sistem manajemen perpustakaan kampus dengan bahasa pemrograman *java*.
2. Menerapkan *array 1D, array 2D, array 3D, arraylist, stack*, dan queue pada satu studi kasus yang sama.
3. Memahami kapan masing-masing struktur data tepat digunakan dan bagaimana operasi dasarnya bekerja.

BAB II
LANDASAN TEORI

2.1 Struktur Data

Struktur data adalah cara menyimpan dan mengatur data di dalam program supaya data tersebut mudah diakses dan diolah. Pemilihan struktur data yang tepat memengaruhi seberapa mudah program ditulis dan seberapa efisien program berjalan. Ada struktur data yang ukurannya tetap seperti array, ada yang ukurannya bisa berubah seperti ArrayList, dan ada yang aturan akses datanya dibatasi seperti Stack dan Queue.

2.2 Array

Array adalah kumpulan elemen dengan tipe data yang sama yang disimpan berurutan di memori. Ukuran array ditentukan saat array dibuat dan tidak bisa diubah setelahnya. Setiap elemen diakses menggunakan indeks yang dimulai dari 0. Jika sebuah array punya 4 elemen, maka indeks yang valid adalah 0 sampai 3.

2.2.1 Array 1 di mensi

Array 1 dimensi adalah bentuk paling sederhana, yaitu satu baris elemen yang diakses dengan satu indeks. Cocok untuk menyimpan daftar data yang jumlahnya tetap, misalnya nama hari atau nama kategori. Contoh deklarasinya adalah String[] nama = new String[4];.

2.2.2 Array 2 dimensi

Array 2 dimensi bisa dibayangkan seperti tabel yang punya baris dan kolom, sehingga elemennya diakses dengan dua indeks, yaitu data[baris][kolom]. Untuk mengisi atau menampilkan seluruh isinya biasanya dipakai perulangan bersarang (nested loop).

2.2.3 Array 3 dimensi

Array 3 dimensi adalah array yang elemennya berupa array 2 dimensi, sehingga dibutuhkan tiga indeks, yaitu data[a][b][c]. Bisa dibayangkan seperti tumpukan beberapa tabel. Dalam kasus ini tiga indeks tersebut mewakili lantai, rak, dan posisi buku.

2.3 ArrayList

ArrayList adalah kelas di paket java.util yang fungsinya mirip array tetapi ukurannya bisa berubah otomatis ketika data ditambah atau dihapus. Method yang sering dipakai antara lain

add() untuk menambah elemen, get() untuk mengambil elemen berdasarkan indeks, remove() untuk menghapus, size() untuk mengetahui jumlah elemen, contains() untuk mengecek apakah suatu elemen ada, dan isEmpty() untuk mengecek apakah list kosong. Di dalamnya ArrayList tetap memakai array, dan array tersebut dibuat ulang dengan ukuran lebih besar ketika sudah penuh.

2.4 Stack

Stack adalah struktur data dengan prinsip **LIFO (Last In First Out)**, yaitu elemen yang dimasukkan paling akhir akan keluar paling awal. Gambarannya seperti tumpukan piring: piring baru diletakkan di atas dan yang diambil juga piring paling atas. Operasi utamanya adalah push() untuk memasukkan elemen ke puncak, pop() untuk mengambil sekaligus menghapus elemen puncak, peek() untuk melihat elemen puncak tanpa menghapusnya, dan isEmpty() untuk mengecek apakah stack kosong. Di Java tersedia kelas java.util.Stack.

2.5 Queue

Queue atau antrean adalah struktur data dengan prinsip **FIFO (First In First Out)**, yaitu elemen yang masuk paling awal akan keluar paling awal. Gambarannya seperti antrean di loket. Operasi utamanya adalah offer() untuk memasukkan elemen ke belakang antrean, poll() untuk mengambil sekaligus menghapus elemen paling depan, dan peek() untuk melihat elemen paling depan tanpa menghapusnya. Di Java, Queue adalah interface, dan salah satu kelas yang mengimplementasikannya adalah LinkedList.

2.6 Ringkasan Perbandingan

Tabel 2. 1 Perbandingan struktur data yang digunakan

Struktur Data	Ukuran	Cara Akses	Dipakai di Program untuk
Array 1D	Tetap	Indeks `[i]`	Daftar kategori dan nama hari
Array 2D	Tetap	Indeks `[i][j]`	Jumlah peminjaman per kategori per hari
Array 3D	Tetap	Indeks `[i][j][k]`	Lokasi buku (lantai, rak, posisi)
ArrayList	Dinamis	Indeks / iterasi	Daftar buku
Stack	Dinamis	LIFO (elemen teratas)	Riwayat transaksi
Queue	Dinamis	FIFO (elemen terdepan)	Antrean mahasiswa

BAB III
PERANCANGAN PROGRAM

3.1 Gambaran Sistem

Program ini dipakai oleh petugas perpustakaan untuk mengelola koleksi buku dan proses peminjaman. Fitur yang tersedia adalah mengelola buku (tambah, tampil, cari, hapus), melihat kategori, melihat statistik peminjaman mingguan, melihat dan mencari lokasi buku di rak, mencatat peminjaman dan pengembalian beserta riwayatnya, membatalkan transaksi terakhir, dan mengatur antrean mahasiswa.

3.2 Pemetaan kebutuhan ke struktur data

Tabel 3. 1 Pemetaan kebutuhan program ke struktur data

No	Kebutuhan	Struktur Data	Variabel di Program
1	Daftar kategori buku	Array 1D	String[] kategori
2	Jumlah peminjaman per kategori dan hari	Array 2D	int[][] peminjaman
3	Lokasi buku berdasarkan lantai, rak, posisi	Array 3D	String[][][] lokasi
4	Daftar buku yang bisa bertambah dan berkurang	ArrayList	ArrayList<Buku> daftarBuku
5	Riwayat transaksi yang bisa dibatalkan dari yang terakhir	Stack	Stack<Transaksi> riwayat
6	Antrean mahasiswa sesuai urutan datang	Queue	Queue<Mahasiswa> antrean

3.3 Struktur Program

Program dibagi menjadi dua package. Package model berisi kelas-kelas data, sedangkan package main berisi kelas utama yang memuat menu dan seluruh logika program.

Tabel 3. 2 Struktur package dan kelas

Package	File	Fungsi
model	Buku.java	Menyimpan data satu buku: ID, judul, penulis, tahun, kategori, lokasi, status, dan peminjam. Objeknya disimpan di ArrayList.
model	Transaksi.java	Menyimpan data satu transaksi (PINJAM atau KEMBALI). Objeknya disimpan di Stack.
model	Mahasiswa.java	Menyimpan data mahasiswa yang antre: nama, NIM, dan ID buku yang diminta. Objeknya disimpan di Queue.
main	PerpustakaanKampus.java	Kelas utama berisi method main, menu, semua struktur data, dan validasi input.

3.4 Rancangan Menu

Tabel 3. 3 Rancangan menu program

Menu Utama	Submenu	Struktur Data
1. Kelola Buku	Tambah buku, tampilkan semua buku, cari buku, hapus buku	ArrayList
2. Kategori Buku	Menampilkan kategori dan jumlah buku tiap kategori	Array 1D
3. Statistik Peminjaman	Tabel peminjaman per kategori per hari, kategori terpopuler, hari tersibuk	Array 2D
4. Lokasi Penyimpanan	Peta rak per lantai, cari lokasi buku, lihat isi satu posisi	Array 3D
5. Transaksi & Riwayat	Pinjam, kembalikan, lihat transaksi terakhir, batalkan transaksi terakhir, tampilkan riwayat	Stack
6. Antrean Peminjaman	Tambah ke antrean, lihat terdepan, layani terdepan, tampilkan antrean	Queue
0. Keluar	Menutup program	-

Tampilan menu utama pada console dapat dilihat pada gambar 3.1



Gambar 3. 1 Tampilan menu utama

3.5 Alur Proses

3.5.1 Alur Peminjaman Buku

1. Petugas mengisi nama mahasiswa, NIM, dan ID buku yang dipinjam.
2. Program mencari buku di ArrayList berdasarkan ID. Jika tidak ada, proses berhenti.
3. Petugas memilih hari transaksi.
4. Program mengecek status buku. Jika masih dipinjam orang lain, peminjaman ditolak dan mahasiswa disarankan masuk antrean.
5. Jika tersedia, status buku diubah menjadi dipinjam, angka pada array 2D (kategori buku, hari) bertambah satu, dan transaksi dimasukkan ke Stack dengan push().
6. Program menampilkan lokasi buku (lantai, rak, posisi) yang diambil dari array 3D.

3.5.2 Alur Pembatalan Transaksi (*Undo*)

1. Program mengecek apakah Stack kosong. Jika kosong, ditampilkan pesan bahwa tidak ada transaksi yang bisa dibatalkan.
2. Jika tidak kosong, transaksi paling atas diambil dengan pop().
3. Jika transaksinya PINJAM, buku dikembalikan ke status tersedia dan angka di array 2D dikurangi satu.
4. Jika transaksinya KEMBALI, buku kembali berstatus dipinjam oleh peminjam yang sama.

3.5.3 Alur Antrean

1. Mahasiswa yang ingin meminjam didaftarkan ke antrean dengan offer(), sehingga berada di urutan paling belakang.
2. Saat petugas melayani antrean, mahasiswa paling depan diambil dengan poll().
3. Program mencoba memproses peminjaman buku yang diminta. Jika berhasil, mahasiswa selesai dilayani.
4. Jika buku masih dipinjam, mahasiswa dimasukkan lagi ke belakang antrean supaya tidak menghalangi mahasiswa lain.

3.5.4 Aturan Validasi Input

Gambar 3. 2 Aturan validasi input

Input	Aturan
Judul buku	3–100 karakter, wajib mengandung huruf (tidak boleh hanya angka), karakter yang boleh: huruf, angka, spasi, dan simbol . , : ; - ' ( ) / & ! ? + #, tidak boleh ada karakter sama berulang 4 kali atau lebih
Penulis	3–60 karakter, hanya huruf, spasi, titik, koma, apostrof, dan tanda hubung (tanpa angka)
Nama mahasiswa	2–50 karakter, hanya huruf, spasi, titik, koma, apostrof, dan tanda hubung
NIM	Hanya angka, panjang 8–12 digit
ID buku	Huruf B diikuti 3 angka, contoh B001
Tahun terbit	Angka antara 1900 sampai tahun berjalan
Pilihan menu, kategori, hari, lantai, rak, posisi	Harus angka dan berada dalam rentang pilihan yang tersedia
Konfirmasi	Hanya y atau n

Semua input diperiksa menggunakan perulangan while (*true*). Kalau input salah, program menampilkan pesan kesalahan lalu meminta input lagi sampai nilainya benar. Selain itu, buku dengan judul dan penulis yang sama tidak bisa ditambahkan dua kali.

BAB IV
IMPLEMENTASI DAN PEMBAHASAN

4.1 Lingkungan Pengembangan

Gambar 3. 3 Lingkungan pengembangan

Komponen	Keterangan
Bahasa pemrograman	Java (JDK 8 ke atas, diuji dengan JDK 21)
Editor / IDE	[isi dengan IDE yang dipakai, misalnya IntelliJ IDEA / NetBeans / VS Code]
Sistem operasi	[isi dengan sistem operasi yang dipakai]
Jenis program	Aplikasi console
Repositori GitHub	https://github.com/[username]/perpustakaan-kampus

Untuk menjalankan program, seluruh *file* dikompilasi lalu kelas main.PerpustakaanKampus dijalankan dengan perintah berikut;

```
javac -d out src/model/*.java src/main/*.java
java -cp out main.PerpustakaanKampus
```

Gambar 4. 1 perintah CLI

Saat program pertama kali dijalankan, sudah terisi 8 buku contoh (B001 sampai B008) dan data statistik peminjaman contoh, supaya semua fitur bisa langsung dicoba tanpa harus input data dulu.

4.2 Implementasi Array 1 Dimensi : Kategori Buku

Daftar kategori disimpan dalam array String berukuran 4. Array nama hari juga dibuat dengan cara yang sama karena akan dipakai sebagai label kolom pada array 2 dimensi.

```
// 1. array 1D: kategori buku
static String[] kategori = {"Pemrograman", "Matematika", "Jaringan Komputer", "Basis Data"};

static String[] hari = {"Senin", "Selasa", "Rabu", "Kamis", "Jumat", "Sabtu", "Minggu"};

static final int LANTAI = 3;
static final int RAK = 4;
static final int POSISI = 5;
```

Gambar 4. 2 array ID : kategori buku

Setiap buku menyimpan nomor indeks kategorinya (0 sampai 3), bukan nama kategorinya. Nama kategori diambil dari array dengan kategori[indeks] ketika akan ditampilkan. Menu Kategori Buku menghitung jumlah buku tiap kategori dengan perulangan seperti berikut.

```
// ================= KATEGORI (Array 1D) =================

static void tampilKategori() {
    System.out.println("\n--- KATEGORI BUKU ---");
    for (int i = 0; i < kategori.length; i++) {
        int jumlah = 0;
        for (Buku b : daftarBuku) {
            if (b.getKategori() == i) jumlah++;
        }
        System.out.printf("%d. %-20s : %d buku%n", i + 1, kategori[i], jumlah);
    }
}
```

Gambar 4. 3 tampilKategori : buku

```
--- KATEGORI BUKU ---
1. Pemrograman          : 3 buku
2. Matematika           : 2 buku
3. Jaringan Komputer    : 2 buku
4. Basis Data           : 2 buku
```

Gambar 4. 4 Tampilan menu kategori buku

4.3 Implementasi Array 2 Dimensi: Statistik Peminjaman

Jumlah peminjaman disimpan dalam array 2 dimensi berukuran 4 × 7. Baris mewakili kategori dan kolom mewakili hari (aenin sampai minggu), sehingga peminjaman[1][3] berarti jumlah peminjaman buku Matematika pada hari kamis.

```
// 2. array 2D: jumlah peminjaman
static int[][] peminjaman = new int[kategori.length][hari.length];
```

Gambar 4. 5 array 2D: jumlah peminjaman

Nilai pada array ini bertambah setiap kali ada peminjaman yang berhasil. Baris yang dipakai adalah kategori buku yang dipinjam dan kolomnya adalah hari yang dipilih petugas, seperti pada method pinjam() berikut;

```
// dipakai juga waktu layani antrean
static boolean pinjam(Buku b, String nama, String nim, int h) {
    if (!b.isTersedia()) return false;

    b.pinjam(nama, nim);
    peminjaman[b.getKategori()][h]++;
    riwayat.push(new Transaksi("PINJAM", b, nama, nim, h));

    System.out.println("Berhasil: \"" + b.getJudul() + "\" dipinjam " + nama + " hari " + hari[h] + ".");
    System.out.println("Ambil bukunya di " + b.getLokasiTeks());
    return true;
}
```

Gambar 4. 6 static boolean pinjam

Untuk menampilkan tabel, dipakai perulangan bersarang. Perulangan luar berjalan per kategori, perulangan dalam berjalan per hari. Di dalam perulangan yang sama dihitung total tiap kategori (jumlah satu baris) dan total tiap hari (jumlah satu kolom), lalu kategori terpopuler dan hari tersibuk dicari dari total tersebut.

```
for (int i = 0; i < kategori.length; i++) {
    int totalKategori = 0;
    System.out.printf("| %-17s |", kategori[i]);
    for (int j = 0; j < hari.length; j++) {
        System.out.printf(" %5d |", peminjaman[i][j]);
        totalKategori += peminjaman[i][j];
        totalHari[j] += peminjaman[i][j];
    }
    System.out.printf(" %5d |%n", totalKategori);
    totalSemua += totalKategori;
    if (totalKategori > maxKategori) {
        maxKategori = totalKategori;
        terpopuler = i;
    }
}
```

Gambar 4. 7 total kategori

Untuk mencetak isi tabel, digunakan perulangan bersarang. Perulangan luar berjalan per kategori (i), sedangkan perulangan dalam berjalan per hari (j). Di dalam perulangan ini, nilai peminjaman dicetak menggunakan System.out.printf, lalu dihitung total per kategori (jumlah satu baris) serta akumulasi total per hari (jumlah satu kolom). Selain itu, dilakukan pengecekan kondisi untuk mencari kategori dengan total peminjaman tertinggi guna menentukan kategori terpopuler.

```
--- STATISTIK PEMINJAMAN MINGGUAN ---
+-------------------+-------+-------+-------+-------+-------+-------+-------+-------+
| Kategori \ Hari   | Sen   | Sel   | Rab   | Kam   | Jum   | Sab   | Min   | Total |
+-------------------+-------+-------+-------+-------+-------+-------+-------+-------+
| Pemrograman       |     5 |     7 |     6 |     8 |     4 |     2 |     1 |    33 |
| Matematika        |     3 |     2 |     4 |     3 |     5 |     1 |     0 |    18 |
| Jaringan Komputer |     2 |     4 |     3 |     5 |     6 |     2 |     1 |    23 |
| Basis Data        |     4 |     3 |     5 |     4 |     3 |     1 |     0 |    20 |
+-------------------+-------+-------+-------+-------+-------+-------+-------+-------+
| Total per hari    |    14 |    16 |    18 |    20 |    18 |     6 |     2 |    94 |
+-------------------+-------+-------+-------+-------+-------+-------+-------+-------+
Kategori terpopuler : Pemrograman (33 peminjaman)
Hari tersibuk       : Kamis (20 peminjaman)
```

Gambar 4. 8 output dari statistik peminjaman mingguan

Dari data contoh, kategori Pemrograman paling banyak dipinjam (33 peminjaman) dan hari Kamis paling ramai (20 peminjaman).

4.4 Implementasi Array 3 Dimensi: Lokasi Penyimpanan

Lokasi buku disimpan dalam array 3 dimensi berukuran 3 × 4 × 5, yaitu 3 lantai, 4 rak per lantai, dan 5 posisi per rak, jadi total ada 60 posisi. Setiap sel menyimpan ID buku yang ada di posisi tersebut, atau null jika kosong.

```
static final int LANTAI = 3;
static final int RAK = 4;
static final int POSISI = 5;
```

Gambar 4. 9 static final lokasi penyimpanan

Ketika buku baru ditambahkan, method taruhDiRak() mencari posisi kosong pertama. Pencarian dimulai dari lantai yang sesuai dengan kategori buku (kategori ke-n mulai dari lantai ke-n), dan kalau lantai itu penuh pencarian dilanjutkan ke lantai berikutnya. Method ini mengembalikan false bila semua 60 posisi sudah terisi.

```
// taruh buku di slot kosong pertama, mulai dari lantai sesuai kategorinya
static boolean taruhDiRak(Buku b) {
    int mulai = b.getKategori() % LANTAI;
    for (int k = 0; k < LANTAI; k++) {
        int l = (mulai + k) % LANTAI;
        for (int r = 0; r < RAK; r++) {
            for (int p = 0; p < POSISI; p++) {
                if (lokasi[l][r][p] == null) {
                    lokasi[l][r][p] = b.getId();
                    b.setLokasi(l, r, p);
                    return true;
                }
            }
        }
    }
    return false;
}
```

Gambar 4. 10 static boolean taruhDiRak

Untuk mencari lokasi sebuah buku, program memeriksa seluruh posisi dengan tiga perulangan bersarang sampai ID yang dicari ketemu.

```
static void cariLokasi() {
    String id = inputId("ID buku");
    for (int l = 0; l < LANTAI; l++) {
        for (int r = 0; r < RAK; r++) {
            for (int p = 0; p < POSISI; p++) {
                if (id.equals(lokasi[l][r][p])) {
                    Buku b = cariById(id);
                    System.out.println("Buku " + id + " (" + b.getJudul() + ")");
                    System.out.println("Lokasi: Lantai " + (l + 1) + ", Rak " + (r + 1) + ", Posisi " + (p + 1));
                    return;
                }
            }
        }
    }
    System.out.println("Buku " + id + " tidak ditemukan di rak.");
}
```

Gambar 4. 11 static void cariLokasi

```
=== LANTAI 1 ===
        Pos 1   Pos 2   Pos 3   Pos 4   Pos 5
Rak 1   [B001 ] [B002 ] [B007 ] [B008 ] [B009 ]
Rak 2   [-    ] [-    ] [-    ] [-    ] [-    ]
Rak 3   [-    ] [-    ] [-    ] [-    ] [-    ]
Rak 4   [-    ] [-    ] [-    ] [-    ] [-    ]

=== LANTAI 2 ===
        Pos 1   Pos 2   Pos 3   Pos 4   Pos 5
Rak 1   [B003 ] [B004 ] [-    ] [-    ] [-    ]
Rak 2   [-    ] [-    ] [-    ] [-    ] [-    ]
Rak 3   [-    ] [-    ] [-    ] [-    ] [-    ]
Rak 4   [-    ] [-    ] [-    ] [-    ] [-    ]

=== LANTAI 3 ===
        Pos 1   Pos 2   Pos 3   Pos 4   Pos 5
Rak 1   [B005 ] [B006 ] [-    ] [-    ] [-    ]
Rak 2   [-    ] [-    ] [-    ] [-    ] [-    ]
Rak 3   [-    ] [-    ] [-    ] [-    ] [-    ]
Rak 4   [-    ] [-    ] [-    ] [-    ] [-    ]

'-' artinya posisi masih kosong
Terisi: 9 dari 60 posisi
```

Gambar 4. 12 peta rak per lantai

Untuk menampilkan denah penyimpanan, dipakai perulangan bersarang yang terbagi berdasarkan lantai, rak, dan posisi. Perulangan luar berjalan per lantai, perulangan menengah berjalan per rak, dan perulangan dalam berjalan per posisi. Di dalam perulangan tersebut, status setiap posisi dicetak; jika terisi akan menampilkan kode barang (seperti B001), sedangkan jika kosong akan menampilkan tanda [-]. Selain itu, program juga menampilkan informasi rekapitulasi berupa total posisi yang terisi dari keseluruhan kapasitas yang tersedia.

```
ID buku (contoh B001): B001
Buku B001 (Java: Dasar Pemrograman)
Lokasi: Lantai 1, Rak 1, Posisi 1
```

Gambar 4. 13 Hasil pencarian lokasi buku B003

4.5 Implementasi ArrayList: Pengelolaan Buku

Seluruh buku disimpan dalam ArrayList<Buku>. Ukurannya tidak perlu ditentukan di awal karena akan bertambah dan berkurang sendiri sesuai jumlah buku.

```
// 4. arraylist: daftar buku
static ArrayList<Buku> daftarBuku = new ArrayList<>();
```

Gambar 4. 14 arraylist: daftar buku

4.5.1 Menambah Buku

Saat menambah buku, program meminta judul, penulis, tahun, dan kategori. Setiap input divalidasi dulu. Setelah itu program mengecek apakah buku yang sama sudah ada, lalu mencari tempat di rak. Jika ada tempat, buku dimasukkan ke ArrayList dengan add(). Setelah satu buku selesai, program bertanya apakah mau menambah buku lagi, dan perulangan while berlanjut selama jawabannya y.

```
static void tambahBuku() {
    boolean lagi = true;
    while (lagi) {
        System.out.println("\n[Tambah Buku]");
        String judul = inputJudul();
        String penulis = inputNama("Penulis", 3, 60);

        if (sudahAda(judul, penulis)) {
            System.out.println("Buku itu sudah ada di koleksi.");
        } else {
            int tahun = inputAngka("Tahun terbit", 1900, Year.now().getValue());

            System.out.println("Pilih kategori:");
            for (int i = 0; i < kategori.length; i++) {
                System.out.println("  " + (i + 1) + ". " + kategori[i]);
            }
            int kat = inputAngka("Nomor kategori", 1, kategori.length) - 1;

            Buku baru = new Buku(judul, penulis, tahun, kat);
            if (taruhDiRak(baru)) {
                daftarBuku.add(baru);
                System.out.println("Buku ditambahkan dengan ID " + baru.getId()
                        + ", lokasi: " + baru.getLokasiTeks());
            } else {
                System.out.println("Rak sudah penuh, buku tidak bisa ditambahkan.");
                return;
            }
```

Gambar 4. 15 static void tambahBuku

Untuk menambahkan data buku baru, dipakai perulangan while yang berjalan terus selama pengguna ingin menambah data. Di dalam perulangan tersebut, dilakukan proses input judul, penulis, tahun terbit, serta pilihan kategori buku. Selain itu, program juga memeriksa ketersediaan buku menggunakan fungsi validasi dan memeriksa kapasitas rak penyimpanan sebelum buku baru berhasil ditambahkan ke dalam sistem.

```
[Tambah Buku]
Judul buku: Ngoding Dasaar
Penulis: Faa
Tahun terbit: 2020
Pilih kategori:
  1. Pemrograman
  2. Matematika
  3. Jaringan Komputer
  4. Basis Data
Nomor kategori: 1
Buku ditambahkan dengan ID B009, lokasi: Lantai 1, Rak 1, Posisi 5
Tambah buku lagi? (y/n): n
```

Gambar 4. 16 output menambah buku

Untuk menampilkan menu penambahan buku, dipakai proses input data berupa judul, penulis, tahun terbit, dan nomor kategori pilihan. Di dalam proses tersebut, sistem secara otomatis menghasilkan ID unik serta menentukan lokasi penyimpanan berdasarkan lantai, rak, dan posisi, lalu menampilkan konfirmasi berhasil beserta opsi untuk mengulang proses penambahan data.

4.5.2 Menampilkan dan Mencari Buku

Seluruh isi ArrayList ditampilkan dengan perulangan for-each. Pencarian dilakukan dengan memeriksa setiap buku, apakah kata kunci yang diketik ada di judul, penulis, atau sama dengan ID buku (tanpa membedakan huruf besar kecil).

```
[Daftar Buku] total 9 buku
+------+------------------------------+----------------------+-------+--------------------+-----------+
| ID   | Judul                        | Penulis              | Tahun | Kategori           | Status    |
+------+------------------------------+----------------------+-------+--------------------+-----------+
| B001 | Java: Dasar Pemrograman      | Budi Raharjo         | 2020  | Pemrograman        | Dipinjam  |
| B002 | Algoritma dan Struktur Data  | Rinaldi Munir        | 2019  | Pemrograman        | Tersedia  |
| B003 | Kalkulus Jilid 1             | Purcell              | 2018  | Matematika         | Tersedia  |
| B004 | Aljabar Linear Elementer     | Howard Anton         | 2017  | Matematika         | Tersedia  |
| B005 | Jaringan Komputer            | Andrew Tanenbaum     | 2021  | Jaringan Komputer  | Tersedia  |
| B006 | TCP/IP Illustrated           | Richard Stevens      | 2016  | Jaringan Komputer  | Tersedia  |
| B007 | Sistem Basis Data            | Abraham Silberschatz | 2020  | Basis Data         | Tersedia  |
| B008 | Dasar-dasar SQL              | Joe Celko            | 2018  | Basis Data         | Tersedia  |
| B009 | Ngoding Dasaar               | Faa                  | 2020  | Pemrograman        | Tersedia  |
+------+------------------------------+----------------------+-------+--------------------+-----------+
```

Gambar 4. 17 Tampilan daftar buku

```
Kata kunci (judul/penulis/ID): Faa
B009 | Ngoding Dasaar | Faa | Pemrograman | Tersedia
```

Gambar 4. 18 hasil pencarian dengan kata kunci “dasar”

Untuk menampilkan hasil pencarian, dipakai proses pengecekan berdasarkan kata kunci berupa judul, penulis, atau ID buku. Di dalam proses tersebut, sistem melakukan pencocokan data dari daftar koleksi, lalu menampilkan informasi detail buku yang ditemukan meliputi ID, judul, penulis, kategori, serta status ketersediaannya.

4.6 Implementasi Stack: Riwayat Transaksi

Setiap peminjaman dan pengembalian dicatat sebagai objek Transaksi lalu dimasukkan ke Stack. Karena sifatnya LIFO, transaksi yang paling akhir berada di puncak sehingga mudah dilihat dan dibatalkan.

```
// 5. stack: riwayat transaksi
static Stack<Transaksi> riwayat = new Stack<>();
```

Gambar 4. 19 static: riwayat transaksi

Transaksi PINJAM dimasukkan dengan push() di dalam method pinjam() (lihat potongan kode pada subbab 4.3). Transaksi KEMBALI dimasukkan dengan cara yang sama di method kembalikanBuku().

```
[Peminjaman Buku]
Nama mahasiswa: Yudi
NIM (8-12 digit): 1987920172643
NIM harus 8-12 digit.
NIM (8-12 digit): 19876154
ID buku yang dipinjam (contoh B001): B007
Pilih hari:
  1. Senin
  2. Selasa
  3. Rabu
  4. Kamis
  5. Jumat
  6. Sabtu
  7. Minggu
Nomor hari: 3
Berhasil: "Sistem Basis Data" dipinjam Yudi hari Rabu.
Ambil bukunya di Lantai 1, Rak 1, Posisi 3
```

Gambar 4. 20 peminjaman buku B007

```
[Pengembalian Buku]
ID buku yang dikembalikan (contoh B001): B007
Berhasil: "Sistem Basis Data" dikembalikan oleh Yudi.
Taruh di Lantai 1, Rak 1, Posisi 3
```

Gambar 4. 21 pengembalian buku B007

Menu Lihat Transaksi Terakhir memakai peek() sehingga transaksi tidak dihapus dari Stack. Sedangkan menu Tampilkan Semua Riwayat membaca isi Stack dari indeks terakhir ke indeks pertama supaya transaksi terbaru muncul di paling atas.

```
static void lihatTerakhir() {
    if (riwayat.isEmpty()) {
        System.out.println("Belum ada transaksi.");
    } else {
        System.out.println("Transaksi terakhir:");
        System.out.println(riwayat.peek());
    }
}
```

Gambar 4. 22 static void lihatTerakhr

```
[Riwayat Transaksi] total 2
#002 | PINJAM  | B007 - Sistem Basis Data | Yudi (19876154) | 25-10-2026 20:13:16  <-- TOP
#001 | PINJAM  | B001 - Java: Dasar Pemrograman | Faa (12345678) | 25-10-2026 20:10:53
```

Gambar 4. 23 riwayat transaksi

Untuk membatalkan transaksi terakhir dipakai pop(). Efek pembatalan bergantung pada jenis transaksinya. Jika transaksi PINJAM dibatalkan, buku kembali tersedia dan angka di array 2D dikurangi satu supaya statistik tetap benar. Jika transaksi KEMBALI dibatalkan, buku kembali berstatus dipinjam oleh orang yang sama. Ada juga pengecekan daftarBuku.contains(b) untuk berjaga-jaga kalau buku dari transaksi itu sudah dihapus dari koleksi.

```java
static void batalkanTerakhir() {
    if (riwayat.isEmpty()) {
        System.out.println("Tidak ada transaksi yang bisa dibatalkan.");
        return;
    }
    Transaksi t = riwayat.pop();
    Buku b = t.getBuku();

    if (!daftarBuku.contains(b)) {
        System.out.println("Transaksi #" + t.getNomor() + " dibuang, tapi tidak bisa dibatalkan karena bukunya sudah dihapus.");
    } else if (t.getJenis().equals("PINJAM")) {
        b.kembalikan();
        if (peminjaman[b.getKategori()][t.getHari()] > 0) {
            peminjaman[b.getKategori()][t.getHari()]--;
        }
        System.out.println("Peminjaman \"" + b.getJudul() + "\" oleh " + t.getNama() + " dibatalkan.");
    } else {
        b.pinjam(t.getNama(), t.getNim());
        System.out.println("Pengembalian \"" + b.getJudul() + "\" dibatalkan, buku kembali dipinjam " + t.getNama() + ".");
    }
}
```

Gambar 4. 24 static void batalkanTerakhir

```
Pengembalian "Sistem Basis Data" dibatalkan, buku kembali dipinjam Yudi.

--- TRANSAKSI & RIWAYAT ---
1. Pinjam buku
2. Kembalikan buku
3. Lihat transaksi terakhir
4. Batalkan transaksi terakhir
5. Tampilkan semua riwayat
0. Kembali
Pilih: 5

[Riwayat Transaksi] total 2
#002 | PINJAM  | B007 - Sistem Basis Data | Yudi (19876154) | 25-10-2026 20:13:16  <-- TOP
```

Gambar 4. 25 pembatalan transaksi terakhir(pengembalian)

4.7 Implementasi Queue: Antrean Mahasiswa

Antrean mahasiswa memakai Queue<Mahasiswa> yang diisi dengan objek LinkedList. Mahasiswa yang mendaftar masuk lewat offer() di belakang antrean, dan dilayani lewat poll() dari depan antrean, sehingga yang datang lebih dulu dilayani lebih dulu.

```java
// 6. queue: antrean mahasiswa
static Queue<Mahasiswa> antrean = new LinkedList<>();
```

Gambar 4. 26 queue: antrean mahasiswa

```
static void tambahAntrean() {
    String nama = inputNama("Nama mahasiswa", 2, 50);
    String nim = inputNim();
    String id = inputId("ID buku yang diinginkan");

    Buku b = cariById(id);
    if (b == null) {
        System.out.println("Buku tidak ditemukan, tidak jadi masuk antrean.");
    } else {
        antrean.offer(new Mahasiswa(nama, nim, id));
        System.out.println(nama + " masuk antrean nomor " + antrean.size()
                + " untuk buku \"" + b.getJudul() + "\".");
    }
}
```

Gambar 4. 27 static void tambahAntrean

Saat melayani antrean, mahasiswa terdepan diambil dengan poll() lalu dicoba dipinjamkan buku yang diminta. Jika bukunya masih dipinjam orang lain, mahasiswa tersebut dimasukkan lagi ke belakang antrean dengan offer(). Cara ini kami pilih supaya satu mahasiswa yang bukunya belum tersedia tidak menahan mahasiswa lain yang bukunya sudah bisa dipinjam.

```
static void layaniAntrean() {
    Mahasiswa m = antrean.poll();
    if (m == null) {
        System.out.println("Antrean kosong.");
        return;
    }
    System.out.println("Melayani: " + m.getNama() + " (" + m.getNim() + ")");

    Buku b = cariById(m.getIdBuku());
    if (b == null) {
        System.out.println("Bukunya sudah tidak ada di koleksi, mahasiswa keluar dari antrean.");
        return;
    }
    int h = pilihHari();
    if (!pinjam(b, m.getNama(), m.getNim(), h)) {
        // buku masih dipinjam, mahasiswa pindah ke belakang antrean
        antrean.offer(m);
        System.out.println("Buku \"" + b.getJudul() + "\" masih dipinjam, "
                + m.getNama() + " dipindah ke belakang antrean.");
    }
}
```

Gambar 4. 28 static void layaniAntrean

Contoh berikut dijalankan setelah buku B002 dipinjam oleh Yudi. Faa mendaftar lebih dulu untuk B002.

```
--- ANTREAN PEMINJAMAN ---
1. Tambah mahasiswa ke antrean
2. Lihat antrean terdepan
3. Layani antrean terdepan
4. Tampilkan semua antrean
0. Kembali
Pilih: 1
Nama mahasiswa: Faa
NIM (8-12 digit): 109872635
ID buku yang diinginkan (contoh B001): B002
Faa masuk antrean nomor 1 untuk buku "Algoritma dan Struktur Data".

--- ANTREAN PEMINJAMAN ---
1. Tambah mahasiswa ke antrean
2. Lihat antrean terdepan
3. Layani antrean terdepan
4. Tampilkan semua antrean
0. Kembali
Pilih: 1
Nama mahasiswa: Yudi
NIM (8-12 digit): 109283746
ID buku yang diinginkan (contoh B001): B007
Yudi masuk antrean nomor 2 untuk buku "Sistem Basis Data".

--- ANTREAN PEMINJAMAN ---
1. Tambah mahasiswa ke antrean
2. Lihat antrean terdepan
3. Layani antrean terdepan
4. Tampilkan semua antrean
0. Kembali
Pilih: 4

[Daftar Antrean] total 2
1. Faa             109872635    -> Algoritma dan Struktur Data   <-- FRONT
2. Yudi            109283746    -> Sistem Basis Data
```

Gambar 4. 29 output antrean

Untuk menampilkan menu antrean peminjaman, dipakai struktur menu pilihan yang mengatur proses penambahan data mahasiswa ke dalam antrean. Di dalam proses tersebut, sistem menerima input berupa nama mahasiswa, NIM, dan ID buku yang diinginkan, lalu menempatkannya secara berurutan. Selain itu, program juga menyediakan fitur untuk melihat, melayani, dan menampilkan keseluruhan daftar antrean beserta posisi terdepan (FRONT).

4.8 Implementasi Validasi Input

Validasi dibuat dalam method terpisah untuk setiap jenis input. Contohnya inputJudul() berikut. Perulangan while (true) baru berhenti (lewat return) kalau semua pemeriksaan lolos. Judul boleh mengandung angka karena ada judul buku seperti “Kalkulus Jilid 1”, tetapi wajib ada hurufnya sehingga judul seperti 1111111111 ditolak.

```java
// judul boleh ada angka, tapi harus ada hurufnya (jadi "1111" ditolak)
static String inputJudul() {
    while (true) {
        System.out.print("Judul buku: ");
        String s = sc.nextLine().trim().replaceAll("\\s+", " ");
        if (s.length() < 3 || s.length() > 100) {
            System.out.println("Judul harus 3-100 karakter.");
        } else if (hitungHuruf(s) < 2) {
            System.out.println("Judul harus mengandung huruf, tidak boleh angka saja.");
        } else if (!karakterBoleh(s, true, ".,:;-'()/&!?+#")) {
            System.out.println("Judul mengandung karakter yang tidak diizinkan.");
        } else if (berulang(s)) {
            System.out.println("Judul tidak wajar, ada karakter yang berulang terlalu banyak.");
        } else {
            return s;
        }
    }
}
```

Gambar 4. 30 static string inputJudul

Validasi NIM memeriksa setiap karakter dengan perulangan for untuk memastikan semuanya angka, lalu mengecek panjangnya.

```java
static String inputNim() {
    while (true) {
        System.out.print("NIM (8-12 digit): ");
        String s = sc.nextLine().trim();
        boolean angkaSemua = s.length() > 0;
        for (int i = 0; i < s.length(); i++) {
            if (!Character.isDigit(s.charAt(i))) angkaSemua = false;
        }
        if (!angkaSemua) {
            System.out.println("NIM hanya boleh angka.");
        } else if (s.length() < 8 || s.length() > 12) {
            System.out.println("NIM harus 8-12 digit.");
        } else {
            return s;
        }
    }
}
```

Gambar 4. 31 static string inputNim

```
NIM (8-12 digit): 123
NIM harus 8-12 digit.
NIM (8-12 digit): abc
NIM hanya boleh angka.
NIM (8-12 digit): 19876542
ID buku yang dipinjam (contoh B001): b1
Format ID salah, contoh yang benar: B001
```

Gambar 4. 32 validasi input NIM dan ID buku

Untuk melakukan validasi input, dipakai proses pengecekan kondisi yang ketat terhadap data yang dimasukkan oleh pengguna. Di dalam proses tersebut, sistem memeriksa panjang digit serta jenis karakter pada NIM, dan jika tidak sesuai akan menampilkan pesan kesalahan seperti “NIM harus 8-12 digit” atau “NIM hanya boleh angka”. Selain itu, dilakukan pula validasi format pada ID buku untuk memastikan kecocokan dengan pola yang ditentukan sebelum data diterima.

BAB V
PENGUJIAN

Pengujian dilakukan dengan menjalankan program dan mencoba input yang benar maupun yang salah pada setiap fitur. Hasilnya dirangkum pada tabel berikut;

Gambar 4. 33 Hasil pengujian

No	Skenario	Input	Hasil yang Diharapkan	Hasil
1	Judul hanya angka	1111111111	Ditolak, diminta input ulang	Sesuai. Muncul pesan judul harus mengandung huruf
2	Penulis mengandung angka	Budi123	Ditolak	Sesuai
3	NIM berisi huruf	abc	Ditolak	Sesuai. Muncul pesan NIM hanya boleh angka
4	NIM kurang dari 8 digit	123	Ditolak	Sesuai. Muncul pesan NIM harus 8-12 digit
5	Format ID salah	b1	Ditolak	Sesuai. Muncul contoh format B001
6	Tahun di luar rentang	1800	Ditolak	Sesuai. Muncul pesan harus antara 1900 sampai 2026
7	Pilihan menu bukan angka	x	Ditolak	Sesuai. Muncul pesan harus berupa angka
8	Menambah buku yang sudah ada	Judul dan penulis B001	Ditolak	Sesuai. Muncul pesan buku sudah ada di koleksi

No	Skenario	Input	Hasil yang Diharapkan	Hasil
9	Meminjam buku yang sedang dipinjam	B001 dipinjam dua kali	Peminjaman kedua gagal	Sesuai. Disarankan masuk antrean
10	Menghapus buku yang sedang dipinjam	B001	Ditolak	Sesuai. Muncul nama peminjam
11	Menghapus buku yang tersedia	B002	Buku terhapus dan posisi rak kosong	Sesuai. Rak terisi 7 dari 60 posisi
12	Membatalkan peminjaman (undo)	Pinjam B001 hari Senin, lalu batalkan	Buku tersedia lagi, angka Pemrograman-Senin kembali ke 5	Sesuai
13	Membatalkan transaksi saat riwayat kosong	-	Muncul pesan, program tidak error	Sesuai
14	Melayani antrean saat kosong	-	Muncul pesan, program tidak error	Sesuai
15	Urutan antrean (FIFO)	Siti (B002) lalu Rina (B003)	Siti dilayani lebih dulu	Sesuai. Rina menjadi urutan pertama setelahnya
16	Melayani antrean saat buku masih dipinjam	Siti antre B001 yang sedang dipinjam	Siti pindah ke belakang antrean	Sesuai

Semua skenario pengujian memberikan hasil yang sesuai dengan yang diharapkan.

BAB VI
PENUTUP

6.1 Kesimpulan

Dari pembuatan program Sistem Manajemen Perpustakaan Kampus ini, kami menyimpulkan beberapa hal:

1. Array 1 dimensi cocok untuk data tetap yang jumlahnya sedikit seperti daftar kategori dan nama hari.
2. Array 2 dimensi cocok untuk data berbentuk tabel. Pada program ini dipakai untuk statistik peminjaman per kategori per hari, dan totalnya bisa dihitung dengan perulangan bersarang.
3. Array 3 dimensi cocok untuk data yang punya tiga tingkat, yaitu lantai, rak, dan posisi buku, sehingga lokasi buku bisa langsung diakses dan dicari.
4. ArrayList lebih praktis daripada array biasa untuk daftar buku karena ukurannya menyesuaikan saat buku ditambah atau dihapus.
5. Stack cocok untuk riwayat transaksi yang harus bisa dibatalkan dari yang paling akhir, karena sifat LIFO-nya sama dengan kebutuhan fitur undo.
6. Queue cocok untuk antrean mahasiswa karena sifat FIFO-nya sesuai dengan aturan siapa yang datang lebih dulu dilayani lebih dulu.

Selain itu, validasi input dengan perulangan membuat program lebih aman karena data yang tidak masuk akal, misalnya judul buku yang hanya berisi angka, bisa ditolak sebelum disimpan.

6.2 Saran

Program ini masih bisa dikembangkan, misalnya dengan:

1. Menyimpan data ke file atau database supaya tidak hilang ketika program ditutup.
2. Menambahkan tanggal peminjaman, tanggal jatuh tempo, dan perhitungan denda keterlambatan.
3. Mendukung beberapa eksemplar untuk satu judul buku.
4. Membuat tampilan grafis (GUI) supaya lebih mudah dipakai petugas.

5. Mengganti Stack dengan Deque (misalnya ArrayDeque), karena kelas Stack di Java termasuk kelas lama dan dokumentasi Java menyarankan Deque untuk operasi stack.